

From Vibe Coding to Agentic Engineering

Andrej Karpathy — in conversation with Stephanie Zhan

Sequoia Capital · 29 min · published 2026-04-29

Vibe coding raised the floor for everyone. Agentic engineering is the discipline of going far faster **without dropping the quality bar**.

"I've never felt more behind as a programmer"

- Agentic tools were useful-but-patchy for a year: good chunks of code, frequent corrections.
- **December was the step change** — the corrections stopped, trust went up, and vibe coding took over.
- Anyone whose mental model of AI is "ChatGPT-adjacent" formed it *before* that shift.
- The result: a side-projects folder that will not stop growing.

"I can't remember the last time I corrected it." — 00:47

Software 3.0: the LLM is the computer

- 1.0 explicit code → 2.0 learned weights → 3.0 prompting a programmable interpreter.
- Your context window is the lever over that interpreter.
- Installing OpenClaw isn't a shell script any more — it's *text you paste to your agent*, which then inspects your machine and debugs itself in the loop.
- The programming question becomes: **what is the piece of text I hand my agent?**

"What's in the context window is your lever over the interpreter that is the LLM." —

02:59

MenuGen: the app that shouldn't exist

- Karpathy built an app to photograph a restaurant menu and generate pictures of each dish: OCR, image gen, Vercel deploy.
- The 3.0 version: hand the photo to Gemini, ask Nano Banana to render the dishes onto the menu. One step.
- **All of MenuGen turned out to be spurious** — scaffolding around a model that could just do it.
- New work isn't only "the old thing, faster" — some of it *couldn't exist before* (e.g. LLM-built knowledge bases from unstructured facts).

"This blew my mind, because actually all of my MenuGen is spurious." — 05:53

Verifiability decides what gets automated

- Classical computers automate what you can **specify in code**; LLMs automate what you can **verify**.
- Frontier labs train in giant RL environments → capability peaks in verifiable domains (math, code) and stays ragged elsewhere.
- Two forces, not one: *is it verifiable* **and** *did a lab care enough to put it in the mix* (chess jumped between GPT-3.5 and 4 because the data was added).
- For founders: a verifiable niche the labs ignore is a real edge — you can build the RL environments and fine-tune yourself.

"Ultimately almost everything can be made verifiable to some extent." — 11:01

Jagged intelligence: the car wash problem

- Same model refactors a 100k-line codebase or finds zero-days — then tells you to **walk** to a car wash 50 m away.
- Jaggedness is the signal: you're either inside the RL circuits (you fly) or outside them (you're pulling teeth).
- Practical consequence: stay in the loop, treat models as tools, and find out empirically which circuits your use case lands in.
- Outside them? Fine-tuning is the answer, not better prompting.

"How is it possible that a state-of-the-art model refactors a 100,000-line codebase and yet tells me to walk to this car wash?" — 11:44

Vibe coding vs. agentic engineering

- **Vibe coding raises the floor** — anyone can build software now.
- **Agentic engineering preserves the ceiling** — same security and quality bar as professional software, just far faster.
- Agents are spiky, fallible, stochastic, and extremely powerful; coordinating them *is* the discipline.
- The old "10x engineer" undersells it — people who are good at this go well past 10x.

"You're not allowed to introduce vulnerabilities due to vibe coding." — 16:08

Hiring hasn't caught up

- Puzzle interviews test the previous paradigm.
- A better test: **give someone a large project** and watch them build it with their own tooling.
- E.g. build a secure "Twitter clone for agents", then let ten agents at high reasoning effort try to break it — they shouldn't be able to.
- What you're grading: setup investment, tool fluency, and whether the result survives adversarial agents.

"Hiring has to look like: give me a really big project and see someone implement it."

— 18:20

What stays human: taste, spec, oversight

- Agents have superb recall for API details (`keepdim` vs `axis` , reshape vs permute) — that's the intern's job now.
- You keep the fundamentals, the design, and the **spec**: agents still do things like key user funds off email addresses instead of a stable user ID.
- Work with the agent to write a detailed spec/docs, then let it fill in the blanks under your oversight.
- Code quality is the current weak spot — bloated, copy-pasted, brittle abstractions — because aesthetics were never in the reward.

"You're in charge of the taste, the engineering, the design." — 22:01

Animals vs. ghosts

- We're not building animals shaped by evolution, curiosity, or intrinsic motivation — we're **summoning ghosts** shaped by data and reward functions.
- Pre-training statistics as substrate, RL bolted on top, sharpening both strengths and defects.
- Yelling at them doesn't help. They aren't animal intelligences.
- The value of the framing is calibration: a better model of *what these things are* makes you more competent with them.

"These are jagged forms of intelligence, shaped by data and reward functions." —

23:30

Agent-native infrastructure is missing

- Nearly everything is still written for humans — docs above all.
- The pet peeve: *"Why are people still telling me what to do? What's the thing I should copy-paste to my agent?"*
- Deploying MenuGen was harder than writing it: DNS, dashboards, settings, service menus — all human-shaped.
- The test for agent-native infra: one prompt builds **and** ships the app, with nothing to click.
- Where it goes: agents representing people and organizations, negotiating the details with each other.

"Everything is still fundamentally written for humans and has to be moved around."

— 25:41

What's still worth learning deeply

"You can outsource your thinking, but you can't outsource your understanding." —
27:55

- Understanding is the bottleneck: you can't direct agents on something you don't grasp.
- Knowledge bases and re-projections of information are how you keep loading context into your own head.
- Every new projection onto the same information yields insight — synthetic data generation, for you.
- Direction requires comprehension; LLMs don't excel at understanding, so that job is still uniquely yours.

Takeaways

1. **December 2025 was the discontinuity** — re-evaluate agents if your view predates it.
2. **Program in 3.0**: ship the text you hand an agent, not the scaffolding around a model.
3. **Follow verifiability** — it's what makes a domain tractable, and where the labs aren't looking is your edge.
4. **Expect jaggedness**; stay in the loop and fine-tune when you're outside the circuits.
5. **Own the spec and the taste**; hand off the API details.
6. **Build agent-native** — if a human has to click through a dashboard, it isn't finished.

► Watch the full conversation: <https://www.youtube.com/watch?v=96jN2OCOfLs>

Slides generated from the transcript by [yt-mem-ai](#) — timestamps anchored with semantic search, not invented.